1번 문제 설명 및 코드 설명

- 기말 프로젝트 문제

- 문제 1

1-1. P1.1 스위치를 누르면 1~2초 사이 랜덤 시간 후에 P1.0 led 1초 점등

1-2. led 점등 후P1.1 스위치 누르면 P4.7 led 1초 점등 & P1.0 led 점등 후 몇 초 후에 눌렸는지
세그먼트에 시간 출력 (s단위 / 소수점 셋째 자리까지)

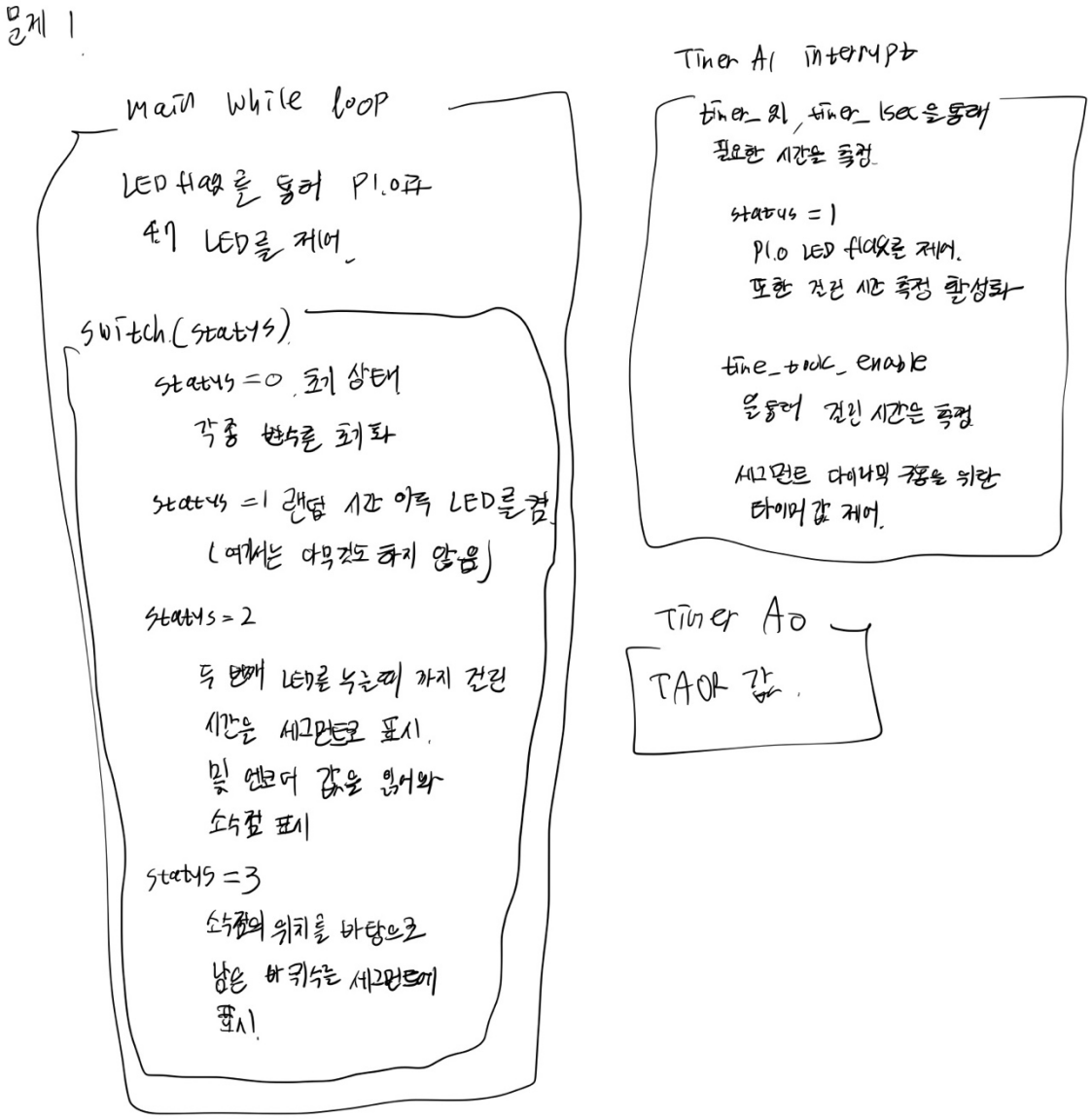
1-3. 엔코더를 시계방향으로 돌리면 소수점이 오른쪽으로 이동, 반시계방향으로 돌리면 왼쪽으로 이동

1-4. P1.1 스위치 push하면 세그먼트에 표시된 바퀴 수만큼 모터 시계방향으로 회전 및 실시간 바퀴수 감소
세그먼트에 출력

1-5. P1.1 스위치 누르면 초기 상태 복귀

코드 설명

- 코드 구조


문제 1
main while loop
LED flag를 통해 P1.0과
4개 LED를 제어.
switch(status)
status = 0 초기 상태
각종 변수를 초기화
status = 1 랜덤 시간 이후 LED를 켬
(여기서는 아무것도 하지 않음)
status = 2
두 번째 LED를 누른 때 까지 걸린
시간을 세그먼트로 표시.
및 엔코더 값을 읽어와
소수점 표시
status = 3
소수점의 위치를 바탕으로
남은 바퀴수를 세그먼트에
표시.
Timer A1 interrupt
timer_ms, timer_1sec을통해
필요한 시간을 측정.
status = 1
P1.0 LED flag를 제어.
또한 걸린 시간 측정 활성화
time_block_enable
을통해 걸린 시간을 측정
세그먼트 다이나믹 구동을 위한
타이머값 제어.
Timer A0
TA0R 값.

Part 1 Interrupt

P1.1을 통해 status값 제어

status = 0

TA0R의 값을 통해 랜덤값 생성

status = 1

P4.7 LED Hex 제어 및

true_hex enable 초기화

status = 2

소수점 위치를 입력받

남은 바퀴수를 세는방법을 제어

encoder Interrupt

status = 2

엔코더 값을 통해 소수점 위치 제어

status = 3

소수점 위치에 따라 두가지 방법으로 남은 바퀴수를 실시간으로 계산

변수 설명

```
volatile int random_value = 0;
```
- TA0R로부터 랜덤(타이머 값)한 값을 받아와 1000~2000 사이의 값으로 만든 뒤 저장하는 변수

```
unsigned int number[10] = { 0xDB, 0x50, 0x1F, 0x5D, 0xD4, 0xCD, 0xCF, 0xD8, 0xDF, 0xDD };
```
- 세그먼트에 쓸 0~9까지의 숫자를 배열로 표현

```
volatile unsigned int status = 0;
```
- 현재 상태(state)를 제어하는 변수

```
unsigned int timer_seg = 0;
```
- 7-세그먼트의 switch 구문을 이용한 다이내믹 구동을 위한 변수

```
unsigned int timer_gl = 0;
```
- 글로벌 타이머. 목표한 랜덤 시간과의 비교에 사용

```
volatile unsigned short led_flag_one = 0;
```
- P1.0 led를 켜고 끄는 flag 변수

```
volatile unsigned short led_flag_two = 0;
```
- P4.7 led를 켜고 끄는 flag 변수

```
unsigned int timer_1sec = 0;
```
- P4.7을 위한 1초 타이머 변수

```
unsigned int time_took = 0;
```
- P1.0이 켜진 뒤 얼마나 시간이 지난 후에 p1.1이 눌렸는지, 그 값을 저장하는 변수

```
unsigned short time_took_enable = 0;
```
- P1.0이 켜진 직후부터 p1.1이 눌렸는지 측정여부를 확인하는 변수

```
int position = 0;
```
- 엔코더 값에 따른 소숫점의 위치

```
int position_cnt = 0;
```
- 엔코더를 통한 값

```
signed int cycle_counter_short = 0;
```

- 소숫점 한자리, 또는 정수만으로 이루어진 바퀴수를 계산하는 변수

```
signed long cycle_counter_precise = 0;
```

- 소숫점 두자리, 세자리로 이루어진 바퀴수를 계산하는 변수

문제를 해결한 주요 logic

1. 랜덤 시간 이후 LED를 1초간 켜고, 시간을 측정하기

1.1) 초기 상태에서 p1.1 스위치를 누르면 현재 TA0R 값에

연산 ((TA0R) % 1000) + 1000)을 하여 random_value 변수에 저장하고, timer_gl을 0으로 초기화.

1.2) timer_gl이 random_value 보다 작으면 led_flag_one = 0, (p1.0 led off) 를 수행. 이후 시간이 지나 timer_gl == random_value 가 되면 led_flag_one = 1 (p1.0 led on)을 수행하고 걸린 시간 측정여부를 결정하는 변수인 time_took_enable = 1을 수행.

1.3) 걸린 시간 측정을 time_took_enable = 1일 경우 timer interrupt가 실행될 때 마다 time_took++을 수행. 즉, 1ms 마다 time_took 변수가 1씩 증가하므로 time_took / 1000 이 걸린 시간(second) 이 됨.

2. 실시간으로 바퀴수 감소 계산 Logic

2.1) 먼저 소숫점의 위치 따라 계산해야할 바퀴수가 달라짐.

2.1.1) 소숫점 아래 한자리까지 바퀴수를 계산하는 경우, 엔코더 값 기준으로 670을

count 하면 1바퀴이므로, 즉 엔코더 값이 67 바뀔 때 마다 0.1바퀴가 이동했다는 뜻이다. cycle_counter_short 변수에 계산해야하는 바퀴수 * 10 의 값을 넣고, 엔코더 값의 카운터인 position_cnt이 67 변화할 때 마다(시계방향, 반시계 방향 모두 position_cnt 값은 증가) cycle_counter_short 값을 1씩 감소시켜준다.

결과적으로 position_cnt 의 값이 67 변화할 때 마다 0.1바퀴씩 남은 바퀴수를 감소시키는 Logic이다.

2.1.2) 소숫점 아래 두자리, 세자리까지 바퀴수를 계산하는 경우 엔코더 값 기준으로, 엔코더 값이 1 변화하면 이는 대략 0.001492 바퀴에 해당한다.

Msp430 프로세서는 float 연산에 취약하므로 이를 정수화시켜 계산한다. 또한 unsigned int자료형의 값의 범위를 넘어서므로 signed long 자료형을 이용한다.

따라서 소숫점 세자리 까지 바퀴수를 계산하기 위해 cycle_counter_precise 라는 signed long type의 변수를 선언하였고, cycle_counter_precise 는 time_took 변수에 소수점의 위치에 따라 1000 또는 10000을 곱한 값이다.

이후 바퀴수를 계산하기 위해 position_cnt의 수치가 1 변화할 때 마다 cycle_counter_precise -1492를 수행한다. 이렇게 하면 position_cnt의 수치가 670 변화하면, 즉 1바퀴 회전을 하면 대략 cycle_counter_precise 는 1,000,000 (백만) 의 값이 감소한다.